



MÔN HỌC: MÁY HỌC TRONG BẢO MẬT MẠNG VÀ HỆ THỐNG

HỆ THỐNG LLM ĐA TÁC TỬ Y KHOA NHẪM NÂNG CAO ĐỘ CHÍNH XÁC THỰC TẾ TRÊN MEDQA-USMLE

Báo cáo giữa kỳ

Giảng viên hướng dẫn: Lê Kim Hùng

Nhóm thực hiện: Nhóm 01

Thành viên:

- Nguyễn Đình Thanh - 250202053
- Ngô Văn Tú - 250202057
- Lê Viết Hưng - 250202040
- Trần Quốc Phong - 250202051

Mục lục

1	Giới thiệu	2
2	Kiến trúc hệ thống	3
3	Thiết kế hệ thống	6
3.1	Các biến thể: V0-V5	6
3.2	Thiết kế RAG	10
3.3	Thiết kế bộ nhớ	13
3.4	Thiết kế bộ kiểm chứng	14
3.5	Tối ưu hoá tiên hoá	15
4	Thiết lập thực nghiệm	15
4.1	Mô hình và prompt	15
4.2	Cấu hình truy hồi	16
4.3	Phần cứng / thiết lập API	16
5	Kết quả đánh giá	17
5.1	Bảng chính (main board): 300 câu đầu của tập test, qwen2.5:7b	17
5.2	So sánh với baseline (so sánh ghép cặp)	17
5.3	Phân tích Thắng/Thua/Hòa	17
5.4	Chi phí và độ trễ	18
6	Phân tích lỗi	18
7	Thảo luận	19
8	Hạn chế	19
9	Kết luận	20
A	Danh mục kiểm tra tái lập	20

Tóm tắt

Nhóm xây dựng và đánh giá một hệ thống LLM đa tác tử (multi-agent) y khoa dạng module với mục tiêu nâng cao *độ chính xác thực tế* (factual accuracy) trên bộ dữ liệu MedQA-USMLE. Hệ thống được tổ chức thành một **thang biến thể** (ablation ladder) gồm năm biến thể (V0 LLM trực tiếp $\rightarrow$ V4 hệ đa tác tử đầy đủ); mỗi biến thể chỉ là cùng một sơ đồ xử lý (StateGraph) của LangGraph, nhưng bật/tắt vài thành phần theo một cấu hình định sẵn tương ứng. Nhờ hai biến thể liền nhau chỉ khác đúng một thành phần, mọi chênh lệch về độ chính xác đều quy thẳng được về thành phần đó. Tất cả biến thể dùng chung một giao diện thống nhất và một bộ đo (độ chính xác kèm khoảng tin cậy bootstrap, tỷ lệ phản hồi không hợp lệ, phân tích Thắng/Thua/Hòa, và kiểm định ghép cặp McNemar), giúp phép so sánh trung thực. Phát hiện thực nghiệm cốt lõi của Nhóm là: trên MedQA **riêng truy hồi (retrieval) không giúp ích** (V1: -1.7 điểm so với V0, không có ý nghĩa thống kê), trong khi **lập luận đa tác tử mới là bước cải thiện sự đầu tiên** (V2: $+5.3$ điểm, McNemar $p=0.044$) - đo trên 300 câu đầu của tập test; bộ kiểm chứng (V3) hiện chưa cho thấy cải thiện thêm so với V2 ở quy mô này. Nhóm trình bày kết quả đo hiện tại, ghi lại đầy đủ kiến trúc, prompt và cấu hình truy hồi để tái lập được, đồng thời liệt kê các phần việc còn lại để đạt tới đánh giá chính thức đầy đủ trên toàn bộ 1273 câu.

1 Giới thiệu

Các mô hình ngôn ngữ lớn (LLM) trả lời đúng nhiều câu hỏi thi y khoa ngay từ đầu, nhưng cũng đưa ra những câu trả lời sai một cách rất tự tin, và không dễ thấy *thành phần kiến trúc* nào thực sự làm giảm những lỗi đó. MedAgents [9] cung cấp bằng chứng ban đầu rằng phối hợp đa tác tử theo dạng hội đồng chuyên gia - các tác tử độc lập đề xuất phân tích, sau đó đi đến đồng thuận - cải thiện lập luận y khoa zero-shot trên MedQA-USMLE; nhưng hệ thống đó gộp đồng thời nhiều thành phần (truy hồi, hội đồng, nhiều vòng thảo luận), khiến khó tách biệt đóng góp riêng của từng yếu tố. Nghiên cứu này đặt câu hỏi thiết kế: *bắt đầu từ một LLM thuần, mỗi thành phần - truy hồi tri thức (RAG), phối hợp đa tác tử (multi-agent), và bước kiểm chứng (verifier) - đóng góp bao nhiêu vào độ chính xác thực tế trên MedQA-USMLE?*

Bài toán. Đầu vào là một câu hỏi trắc nghiệm MedQA-USMLE [1] (một tình huống lâm sàng kèm bốn lựa chọn A-D). Đầu ra bắt buộc là *đúng một* lựa chọn cuối cùng cộng với một giải thích ngắn (≤ 30 từ).

Cách tiếp cận. Nhóm không thực hiện tách rời năm hệ thống riêng biệt. Thay vào đó, mỗi biến thể là *cùng một* bộ khung LangGraph [4], chỉ bật/tắt các nút (node) khác nhau theo một cấu hình định sẵn, và mỗi bậc chỉ bổ sung *đúng một* thành phần so với bậc trước: V0 $\rightarrow$ V1 thêm truy hồi agentic (RAG), V1 $\rightarrow$ V2 tách một tác tử duy nhất thành bốn nút chuyên trách - hiểu ca bệnh, tìm kiếm, lập luận, quyết định (Mục 3.1) - V2 $\rightarrow$ V3 thêm bộ kiểm chứng và bộ nhớ ngắn hạn, còn V3 $\leftrightarrow$ V4 bật/tắt đúng bộ kiểm chứng đó. Thiết kế bốn nút này khác hẳn hội đồng nhiều chuyên gia kiểu MedAgents mà Nhóm đã thử trước khi ổn định trên kiến trúc hiện tại (§3.1). Vì một bộ đo duy nhất chấm cả năm biến thể theo cùng một bộ đánh giá, mọi chênh lệch điểm số đều so sánh được trực tiếp.

Đóng góp của nhóm nghiên cứu.

1. Một kiến trúc đa tác tử điều khiển bằng cấu hình (V0-V5) - việc khám phá ban đầu lấy cảm hứng từ MedAgents [9], nhưng kiến trúc hiện tại là một thiết kế bốn nút riêng của Nhóm (Mục 3.1) - trong đó mỗi cải tiến khác phiên bản trước đó đúng một thành phần (Mục 2-3).

2. Một quy trình đánh giá ghép cặp, tái lập được, có kiểm định ý nghĩa thống kê (Mục 4-5).
3. Kết quả thực nghiệm cho thấy trên MedQA, *cấu trúc lập luận đa tác tử*, chứ không phải *truy hỏi*, mới là yếu tố làm thay đổi độ chính xác - nhất quán với lý giải của MedAgents nhưng lần đầu được tách biệt qua thiết kế ablation có kiểm soát (Mục 5-7).

Trạng thái đo lường. Kiến trúc (V0-V4) đã được xây dựng đầy đủ và kiểm tra chạy được đầu-cuối. Báo cáo này đo V0-V3 trên kiến trúc *hiện tại* (thiết kế bốn nút + luật tin cậy bất đối xứng, §3) ở $n=300/1273$ câu test (§5): V2 là cải thiện có ý nghĩa thống kê đầu tiên so với V0 ($p=0.044$); V1 (retrieval) và V3 (+ bộ kiểm chứng) chưa cho thấy khác biệt có ý nghĩa so với V0. V4 chưa được đo. Mỗi khoảng trống còn lại (đo ở $n=1273$, đo V4) đều được đánh dấu **[CẦN LÀM:]** để người đọc biết chính xác điều gì đã đo và điều gì mới là dự kiến.

2 Kiến trúc hệ thống

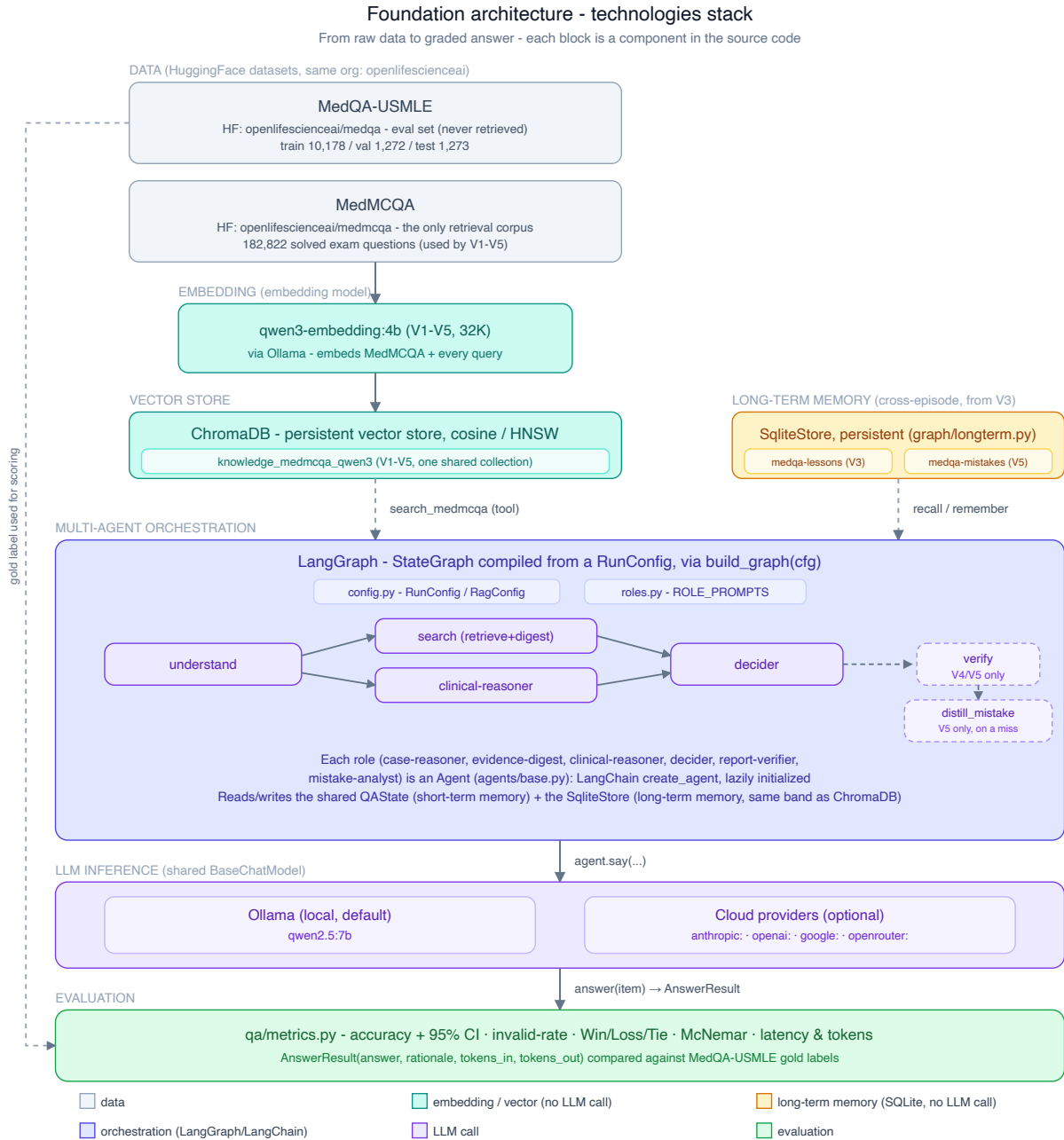
Ý tưởng là tạo một bộ khung ứng dụng duy nhất với các nút được bật/tắt tùy biến thể:

understand $\rightarrow$ (search || reasoning) $\rightarrow$ decide $\rightarrow$ [verify] $\rightarrow$ [distill_mistake]

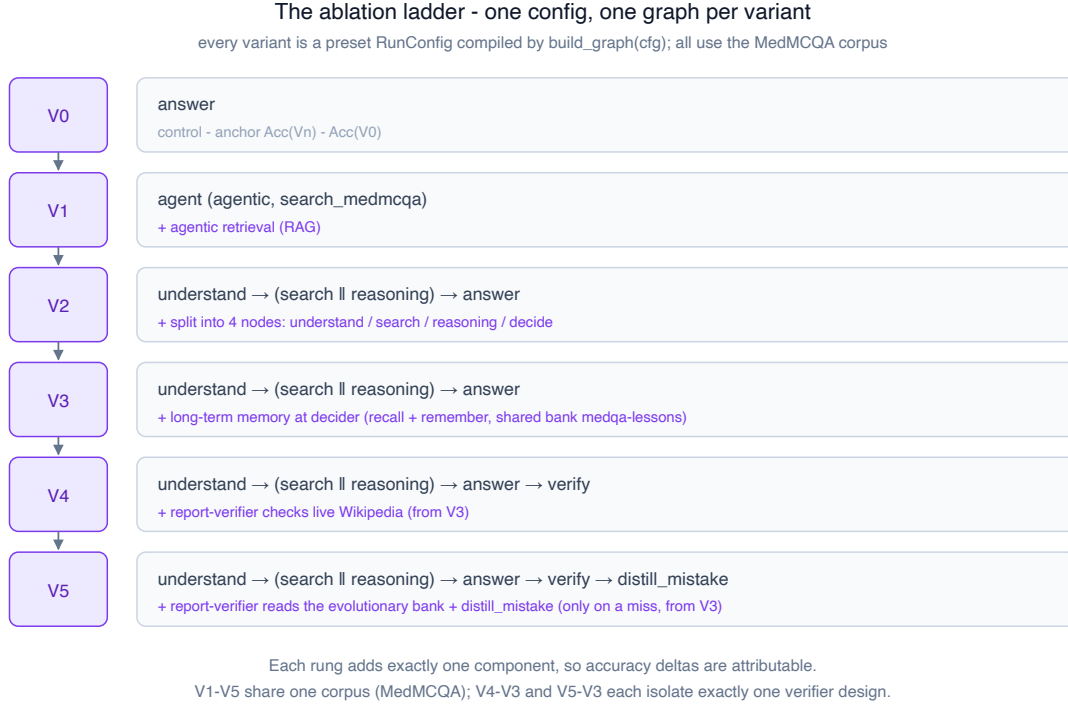
Một biến thể không được tạo bởi một nhánh mã riêng, mà nó phụ thuộc vào một `RunConfig` định sẵn, sau đó được biên dịch thành một `StateGraph` của `LangGraph` [4] qua `build_graph(cfg)`.

Vai trò từng thành phần trong stack (Hình 1).

- **Dữ liệu (HuggingFace datasets): MedQA-USMLE** [1] là bộ đề dùng để đánh giá, không bao giờ dùng để truy hỏi; **MedMCQA** [7] là dữ liệu truy hỏi duy nhất, dùng chung cho toàn bộ V1-V5.
- **Embedding:** một embedder duy nhất qua Ollama cho toàn bộ V1-V5 - **qwen3-embedding:4b** (ngữ cảnh 32K token). V2-V5 dùng một embedder ngữ cảnh ngắn hơn (8192 token) ở một thiết kế trước đó, nhưng đã chuyển sang cùng embedder với V1 sau một A/B nhỏ - xem §3.2.
- **Lưu trữ vector - ChromaDB:** vector persistent database, đo độ tương đồng bằng Cosine Estimation, một bộ sưu tập duy nhất (`knowledge_medmcqa_qwen3`), dùng chung cho V1-V5.
- **Bộ nhớ dài hạn (cross-episode) - SqliteStore:** persistent database cho nhiều round (`data/longterm/`, `graph/longterm.py`), tách biệt với `QASState` (bộ nhớ ngắn hạn trong một câu). Hai namespace độc lập: `medqa-lessons` - bài học chung mà `decider` tự recall/remember (từ V3), và `medqa-mistakes` - kho tiền hoá các ca đã trả lời SAI mà `verify` recall và `distill_mistake` remember (chỉ V5, §3.3).
- **Điều phối đa tác tử - LangGraph:** `StateGraph` biên dịch một `RunConfig` (`config.py`) thành đồ thị các nút (`graph/nodes.py`, `graph/build.py`); mọi nút đọc/ghi qua `QASState` dùng chung (`graph/state.py`) - đây chính là kênh giao tiếp chính của các nodes trong hệ thống (§3.3).
- **Tác tử - LangChain:** mỗi role trong đồ thị (tối đa 6 roles ở V5, xem Hình 1) là một **Agent** (`agents/base.py`), bọc `create_agent` của `LangChain` - khởi tạo không phụ thuộc nhà cung cấp mô hình LLM. Prompt của từng role nằm trong `roles.py`.
- **Suy luận LLM: Ollama** cục bộ (`qwen2.5:7b`) là mặc định; các nhà cung cấp đám mây (`anthropic:`, `openai:`, `google:`, `openrouter:`) gọi bằng interface `BaseChatModel` qua tham số `provider:model` (§4).
- **Đánh giá:** `qa/metrics.py` so `AnswerResult` (đáp án, giải thích, token) với nhãn đúng của MedQA-USMLE, tính độ chính xác kèm khoảng tin cậy bootstrap, tỷ lệ không hợp lệ, Thắng/Thua/Hòa, và McNemar (§5).



Hình 1: Toàn bộ technologies stack, từ dữ liệu thô đến đáp án được chấm điểm.



Hình 2: Thang khảo sát ablation. Mỗi biến thể thêm đúng một khả năng so với biến thể trước.

V1 là ngoại lệ của bộ khung trên: thay vì tách **understand/search/reasoning** thành các nút riêng, nó rút về một nút **agent** duy nhất, tự làm cả bốn việc trong một lượt hội thoại có gọi công cụ (xem §3.2). V2-V5 theo đúng bộ khung bốn nút, với Node 1 (**understand**) sinh ra bản tóm tắt ca bệnh + truy vấn dùng chung cho hai nhánh Node 2 (**search**) và Node 3 (**reasoning**) chạy *đồng thời*, trước khi Node 4 (**answer**, vai **decider**) tổng hợp; V3 không thêm nút nào (**decider** tự đọc/ghi bộ nhớ dài hạn), V4/V5 mỗi biến thể thêm đúng một nút sau **answer** (Node 5 **verify**; V5 thêm cả Node 6 **distill_mistake**).

Bản đồ module. Toàn hệ thống nằm gọn trong một số ít tệp, mỗi tệp một nhiệm vụ: **config.py** (nơi tập trung mọi tùy chỉnh: **RunConfig/RagConfig**), **roles.py** (mọi system prompt), **graph/** (trạng thái, các hàm tạo node, và **build_graph**), **qa/variants.py** (các cấu hình định sẵn V0-V5 và điểm vào công khai **build_variant**), **qa/mcq.py** (định dạng prompt và trích xuất câu trả lời), **qa/metrics.py** (mọi phép đo), **knowledge/** (nạp dữ liệu RAG, embedding, bộ truy hồi), và **agents/base.py** (lớp bọc tác tử khởi tạo rồi, không phụ thuộc nhà cung cấp).

Các tính chất thiết kế.

- **Giao diện thống nhất:** mọi biến thể đều là **answer(item) → AnswerResult(answer, rationale)**, nên một bộ đo duy nhất chấm điểm tất cả như nhau.
- **Khởi tạo trễ / an toàn khi offline:** tác tử được dựng ở lần gọi đầu và kho vector mở ở lần truy hồi đầu, nên việc biên dịch một đồ thị không đụng tới mạng; các bài kiểm thử chạy ngoại tuyến.
- **Mỗi lần một thứ:** thêm một biến thể hay A/B một tham số (mô hình, embedder, kích thước hội đồng, bật/tắt kiểm chứng) là thay đổi cấu hình chứ không phải viết mã mới.

Bảng 1: Các nút thực sự được nối trong mỗi biến thể (tên nút khớp với mã nguồn).

Biến thể	Các nút tác tử	Số lượng tác tử
V0	<code>answer</code>	1
V1	<code>agent</code> (agentic, gọi <code>search_medmcqa</code>)	1
V2	<code>understand</code> → (<code>search</code> <code>reasoning</code>) → <code>answer</code>	4
V3	<code>understand</code> → (<code>search</code> <code>reasoning</code>) → <code>answer</code> + long-term memory	4
V4	V3's graph + <code>verify</code> (Wikipedia-grounded)	5
V5	V3's graph + <code>verify</code> + <code>distill_mistake</code>	6

3 Thiết kế hệ thống

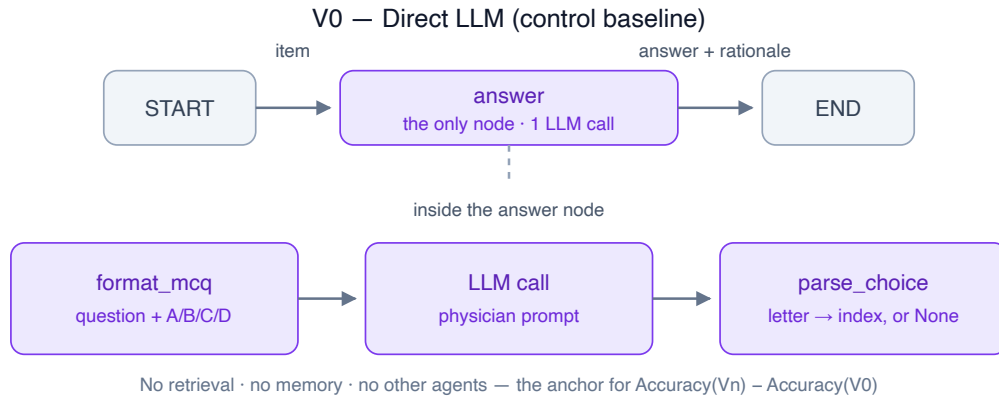
3.1 Các biến thể: V0-V5

Mỗi tác tử là một lớp bọc mỏng, khởi tạo trẻ quanh `create_agent` của LangChain với một `BaseChatModel` không phụ thuộc nhà cung cấp; chỉ cần ghi tên mô hình dưới dạng chuỗi là nó được hiểu thành một mô hình Ollama cục bộ. Mục này giới thiệu từng biến thể theo đúng thứ tự thang ablation - luồng xử lý, các tác tử tham gia, và điều mỗi bậc thêm vào so với bậc trước; §3.2 đi sâu hơn vào các quyết định thiết kế truy hồi dùng chung cho V1-V4.

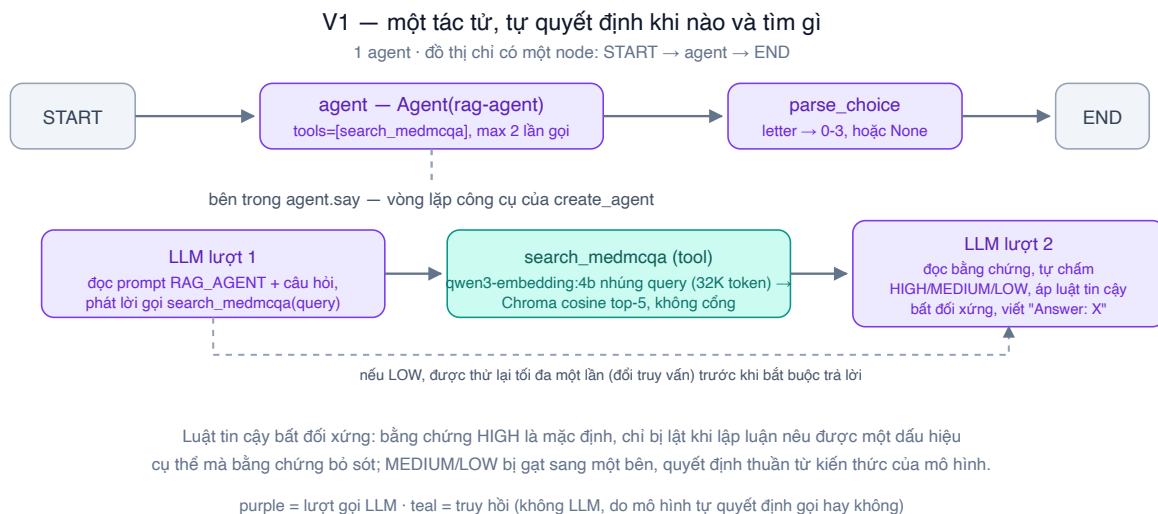
V0 - LLM trực tiếp (đối chứng). Một lượt gọi LLM duy nhất, không truy hồi, không tác tử thứ hai: `format_mcq` dựng prompt (tình huống + A-D), model trả lời, `parse_choice` trích chữ cái. Đây là mốc mà mọi biến thể khác được so sánh vào: Độ chính xác(V_n) – Độ chính xác(V_0).

V1 - một tác tử agentic RAG. Đồ thị rút về một node: `START` → `agent` → `END`. Bên trong, `Agent(rag-agent)` tự quyết định có gọi `search_medmcqa` hay không (tối đa hai lần), tự chấm độ tin cậy của bằng chứng lấy về (HIGH/MEDIUM/LOW), rồi áp luật *tin cậy bất đối xứng* (§3.2) để chốt đáp án - tất cả trong cùng một tác tử, không có node/tác tử riêng cho từng bước.

V2 - bốn nút: hiểu ca bệnh, tìm kiếm, lập luận, quyết định. V2-V4 dùng chung một thiết kế bốn nút, tách đúng bốn việc mà tác tử đơn của V1 làm nối tiếp bên trong nó thành bốn



Hình 3: V0 - một node duy nhất, không truy hồi, không tác tử thứ hai.



Hình 4: V1 - một tác tử agentic tự gọi công cụ, tự chấm độ tin cậy, tự áp luật quyết định.

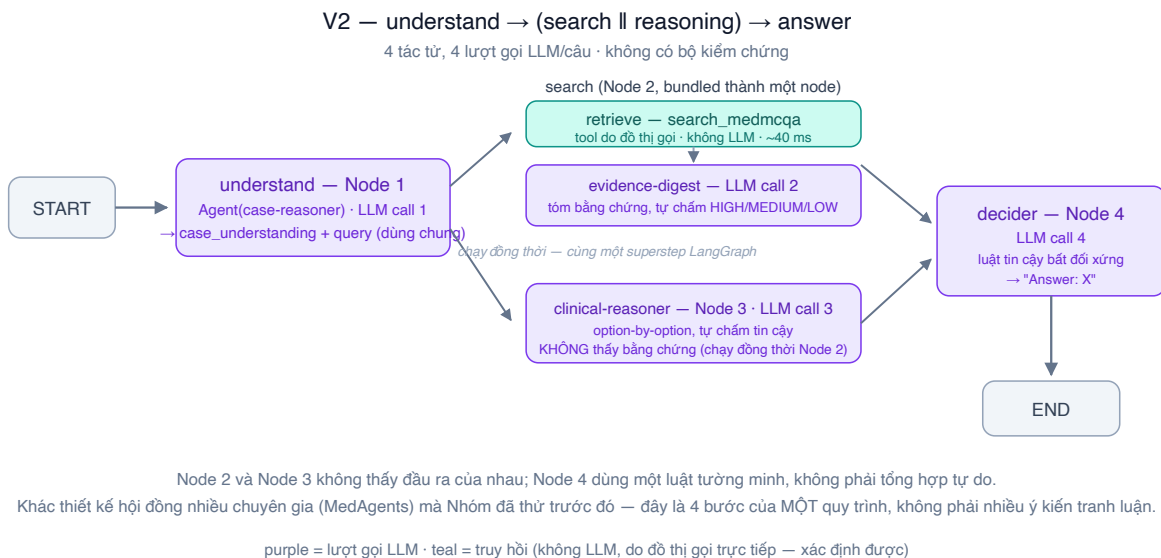
nút riêng, chạy qua LangGraph thay vì một vòng lặp công cụ:

- **Node 1 - case-reasoner (understand):** đọc tình huống một lần, viết ra một bản tóm tắt ca bệnh và một truy vấn tìm kiếm, dùng chung cho hai nhánh sau; không nêu đáp án.
- **Node 2 - search (tìm kiếm + rút gọn):** gọi `search_medmcqa` với truy vấn của Node 1, rồi `evidence-digest` tóm bằng chứng thành một đoạn ngắn kèm mức tin cậy tự đánh giá.
- **Node 3 - clinical-reasoner (reasoning):** lập luận từ CÙNG bản tóm tắt của Node 1, hoàn toàn không thấy bằng chứng của Node 2 (hai nhánh chạy đồng thời); tự chấm mức tin cậy của chính lập luận đó.
- **Node 4 - decider (answer):** nhận hai đầu vào độc lập kèm mức tin cậy riêng, áp luật tin cậy bất đối xứng để chốt đáp án.

4 tác tử, 4 lượt gọi LLM/câu, không có bộ kiểm chứng. Thiết kế này khác thiết kế hội đồng nhiều chuyên gia (kiểu MedAgents [9]) mà Nhóm đã thử trước đó: thay vì nhiều tác tử tranh luận theo nhiều góc nhìn rồi một bác sĩ chính hoà giải, Node 2 và Node 3 tách biệt hoàn toàn, và Node 4 dùng một luật tường minh để quyết nhánh nào được tin.

V3 - V2 KHÔNG có bộ kiểm chứng, nhưng decider có bộ nhớ dài hạn. [CẦN LÀM: Đối định nghĩa so với bản báo cáo trước: V3 từng là "V2 + bộ kiểm chứng"; số liệu 0.650 acc ở §5 mô tả định nghĩa CŨ đó, không còn khớp với V3 hiện tại - xem ghi chú đầu §5.] V3 giữ nguyên bốn nút của V2 (không thêm Node 5), nhưng bật `long_term=True`: Node 4 (decider) tự `recall` bài học từ các câu TƯƠNG TỰ đã gặp trước đó (một *kho chung*, xem §3.3) trước khi chốt đáp án, rồi tự `remember` lại một bài học sau khi chốt - không cần một tác tử/lượt gọi LLM riêng cho việc "nhớ", chỉ là hai lần đọc/ghi kho thêm quanh lượt gọi decider sẵn có. 4 tác tử, 4 lượt gọi LLM/câu.

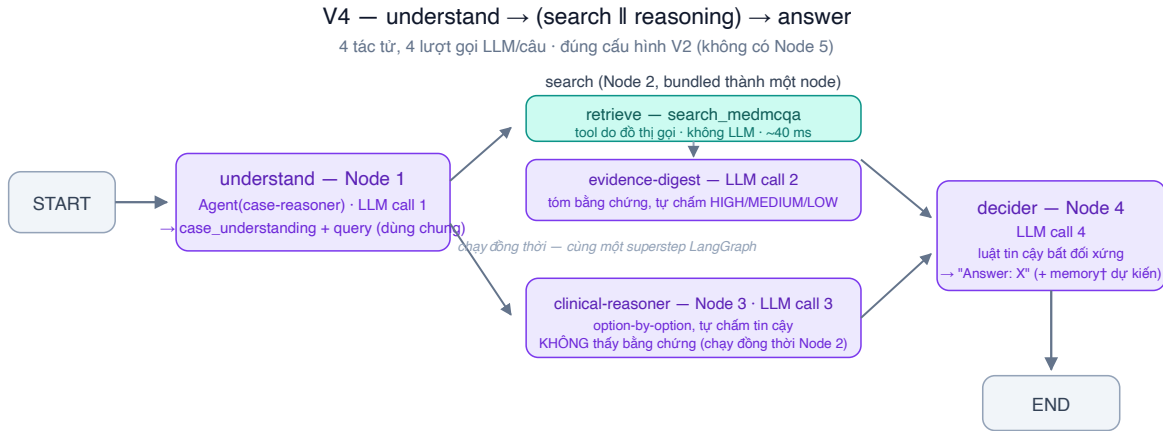
V4 - V3 + bộ kiểm chứng đối chiếu Wikipedia. V4 thêm **Node 5 - report-verifier**: đọc lại báo cáo lập luận của Node 3 và bản tóm tắt ca bệnh của Node 1 (`memory=True`), thăng từ trạng thái đồ thị dùng chung, rồi xác nhận hoặc chỉnh đáp án của Node 4 (§3.4). Khi báo cáo không đủ để quyết, nó gọi `search_wikipedia` - một lệnh gọi Wikipedia *trực tuyến*, độc lập với ngữ liệu MedMCQA cục bộ mà toàn hệ thống còn lại dùng để truy hồi (chính ngữ liệu đó cũng là



Hình 5: V2 - Node 1 phân nhánh đồng thời sang Node 2/Node 3, hội tụ ở Node 4.

nguồn của bộ đề đang được chấm, nên tra lại nó không phải một phép kiểm độc lập thật sự). 5 tác tử, 5 lượt gọi LLM/câu (cộng lệnh gọi Wikipedia không tính là lượt LLM).

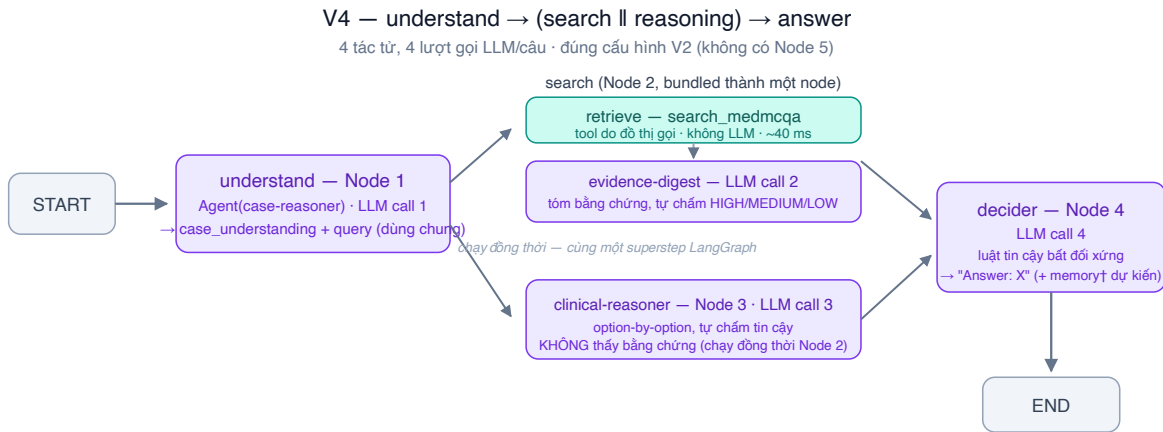
V5 - V3 + bộ kiểm chứng đọc "kho tiến hoá" các ca đã sai. V5 thêm CÙNG Node 5 của V4 nhưng không gọi Wikipedia; thay vào đó nó **recall** từ một kho *riêng biệt* với kho chung của V3 - **kho tiến hoá (medqa-mistakes)**, chỉ chứa bài học từ những câu hệ thống từng trả lời SAI (§3.3). Bộ kiểm chứng không bao giờ biết đáp án đúng của câu đang xử lý - nó chỉ đọc bài học đã đúc kết từ những câu KHÁC, tương tự về chủ đề. V5 thêm **Node 6 - distill_mistake**, chạy SAU **verify**: đây là nút DUY NHẤT trong toàn đồ thị đọc đáp án đúng (`item.answer_idx`) - nếu đáp án cuối cùng SAI, nó gọi một tác tử **mistake-analyst** (được cho biết đáp án đúng) để đúc kết một bài học sửa sai, rồi ghi vào kho tiến hoá cho các câu tương lai; nếu đáp án ĐÚNG, nó không làm gì (không tốn lượt gọi LLM). 6 tác tử, tối đa 6 lượt gọi LLM/câu (5 khi đáp án đúng).



Không có Node 5, nên không đọc case_understanding/clinical_report ngoài Node 4 tự đọc để trả lời.
V4 hiện là đúng RunConfig của V2 (verify=False, memory=False) — không lẫn với ngữ liệu/embedder/số tác tử của bản thiết kế trước.

† kế hoạch: bật memory=True cho V4 để V3/V4 chỉ khác đúng field verify; qa/variants.py chưa cập nhật (xem báo cáo, mục 3.4).

Hình 6: V4 (sơ đồ minh hoạ, chưa cập nhật theo định nghĩa mới - **[CẦN LÀM: vẽ lại docs/diagrams/v4_flow.svg: đồ thị V3 + Node 5 đối chiếu Wikipedia]**) - Node 5 audit đáp án của Node 4, đọc thẳng Node 1/Node 3 từ QASState dùng chung.



Không có Node 5, nên không đọc case_understanding/clinical_report ngoài Node 4 tự đọc để trả lời.
V4 hiện là đúng RunConfig của V2 (verify=False, memory=False) — không lẫn với ngữ liệu/embedder/số tác tử của bản thiết kế trước.

† kế hoạch: bật memory=True cho V4 để V3/V4 chỉ khác đúng field verify; qa/variants.py chưa cập nhật (xem báo cáo, mục 3.4).

Hình 7: V5 (sơ đồ minh hoạ, dùng tạm sơ đồ V4 - **[CẦN LÀM: vẽ docs/diagrams/v5_flow.svg riêng: đồ thị V3 + Node 5 (đọc kho tiền hoá) + Node 6 distill_mistake sau Node 5, nút duy nhất đọc gold]**) - Node 5 không thấy gold; chỉ Node 6 thấy, và chỉ ghi kho khi đáp án cuối cùng sai.

3.2 Thiết kế RAG

V1-V5 nay dùng chung cả *nguồn dữ liệu* lẫn *embedder* truy hồi: **MedMCQA** (182.822 câu thi đã giải) qua **qwen3-embedding:4b** (collection `knowledge_medmcqa_qwen3`, ngữ cảnh 32K token, nên không bị cắt như Query encoder 64 token của MedCPT) - chênh lệch giữa các bậc nay chỉ còn do kiến trúc, không lẫn với việc đổi ngữ liệu hay embedder giữa các bậc. Xem cuối mục này về lý do chọn embedder này và bằng chứng A/B.

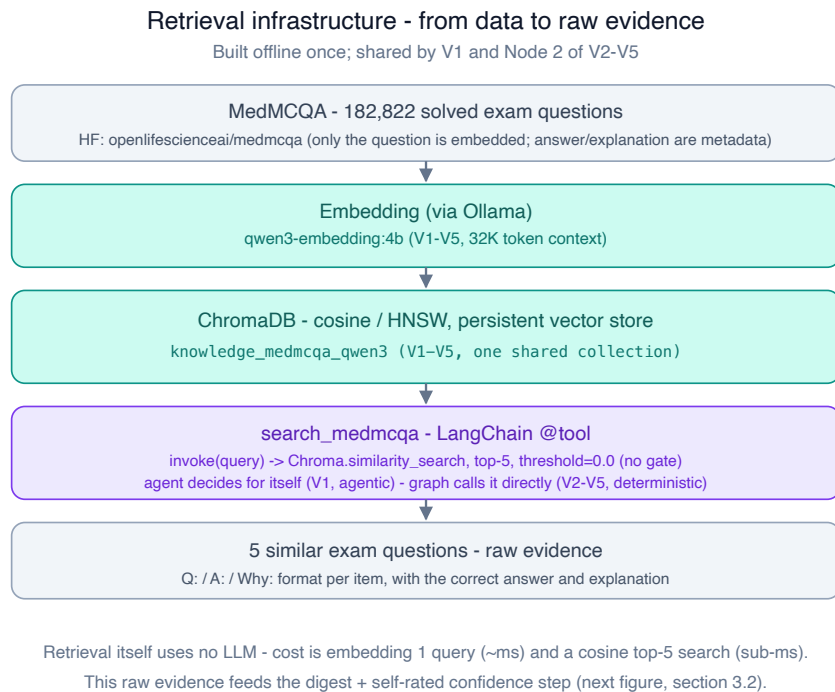
Luồng xử lý của từng biến thể đã trình bày ở §3.1; hai điểm sau là cơ chế truy hồi *dùng chung*, cắt ngang mọi biến thể chứ không riêng cho một bậc nào:

1. **Công cụ truy hồi top- k , không còn cổng số.** `search_medmcqa` luôn trả về $k=5$ câu thi gần nhất kèm đáp án và giải thích (định dạng Q:/A:/Why:), cho mọi biến thể (`threshold=0.0`). Thiết kế trước dùng một *cổng độ tương đồng* bằng số (ngưỡng cosine, hiệu chỉnh riêng theo từng embedder); cổng đó nay được thay bằng **mô hình tự đánh giá**: bước chốt đáp án dùng nhãn tin cậy tự chấm (HIGH/MEDIUM/LOW) của chính tác tử/nút vừa đọc bằng chứng, chứ không so một điểm cosine với một ngưỡng cố định.
2. **Luật quyết định: tin cậy bất đối xứng.** Cùng một luật cho cả bước chốt đáp án của V1 lẫn Node 4 (`decider`, V2-V4): nếu bằng chứng truy hồi được chấm HIGH và rõ ràng nghiêng về một phương án, phương án đó là đáp án mặc định - chỉ bị lật lại khi có một dấu hiệu *cụ thể* trong tình huống mà bằng chứng đã bỏ sót hoặc hiểu sai, không phải một cảm giác chung chung là “có lẽ khác”. Nếu bằng chứng chỉ MEDIUM/LOW hoặc không có, nó bị gạt sang một bên hoàn toàn và đáp án được chọn thuần từ lập luận lâm sàng, đúng như cách V0 quyết định. Luật bất đối xứng có chủ đích: bằng chứng tốt được ưu tiên mặc định, nhưng không tự động thắng một lập luận nên được dấu hiệu cụ thể.

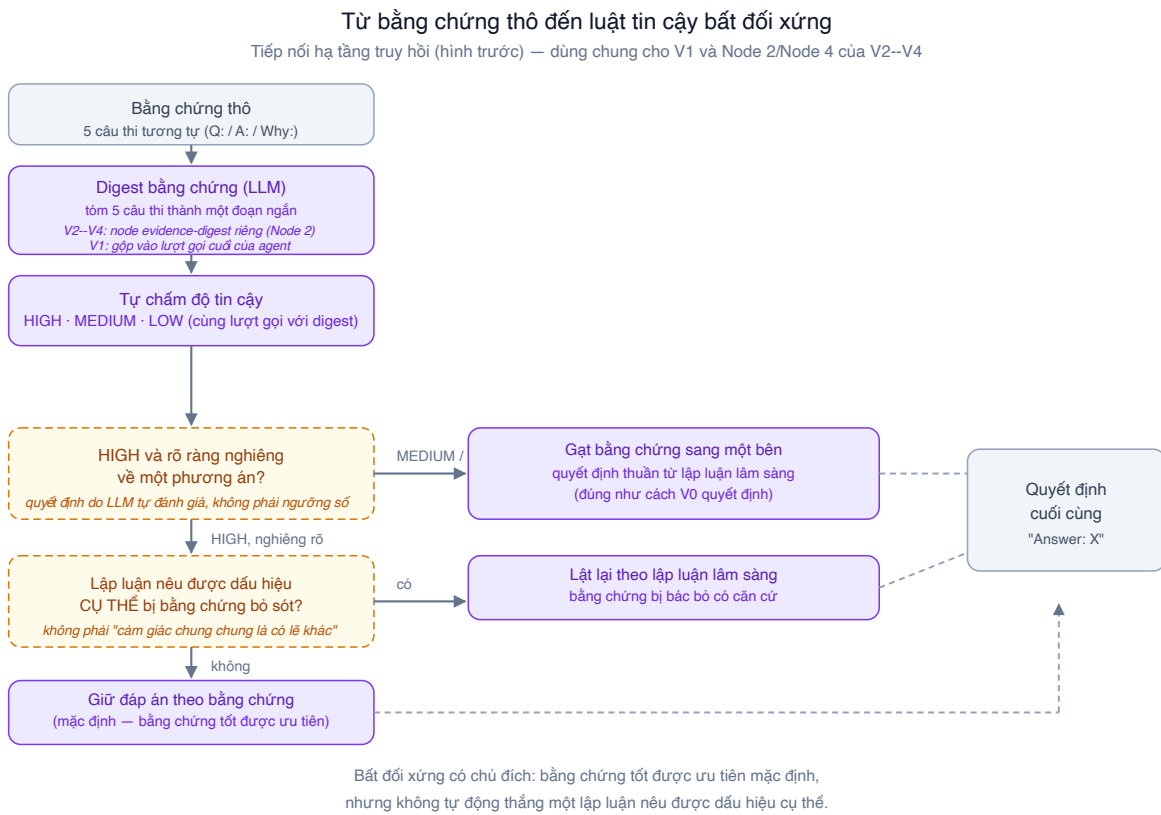
Truy hồi là công cụ do mô hình chọn, chỉ ở V1. V1 để tác tử tự quyết định khi nào và tìm gì, thay vì đồ thị gọi cứng như V2-V4. Đây là chi phí có chủ đích: đo trên cùng tập câu, cách do tác tử điều khiển tốn 3 lần gọi LLM / 542 ký tự sinh ra / 3.78s mỗi câu, so với 2 lần gọi / 51 ký tự / 0.90s khi để đồ thị gọi: một lời gọi công cụ do mô hình chọn cần thêm một lần gọi để xử lý kết quả, và tác tử dùng công cụ không thể bị ép “chỉ trả lời bằng chữ cái” (chỉ dẫn đó triệt tiêu việc gọi công cụ; đo được 0/5), nên đáp án được trích từ phần lý do ngắn thay vì một chữ cái đơn. Nhóm chấp nhận chi phí này vì mục tiêu là V1 thành một tác tử RAG đúng nghĩa; V2-V4 vẫn gọi truy hồi từ node để giữ tính tất định và chi phí thấp.

Trong `agent.say` (vòng lặp công cụ của `create_agent`), *mô hình* tự quyết định gọi công cụ, nên đồ thị chỉ có một node; truy hồi trả về top- k không qua cổng số, và bước quyết định ở lượt 2 dùng nhãn tin cậy tự chấm thay cho một ngưỡng cosine. Chi phí ~ 2 lần sinh của LLM mỗi câu (lượt 1 gọi công cụ, lượt 2 trả lời) là lý do V1 chậm hơn hẳn V0.

Vì sao toàn hệ thống dùng qwen3-embedding:4b. Thiết kế ban đầu dùng một embedder ngữ cảnh 8192 token làm mặc định, nhưng V1 chuyển sang **qwen3-embedding:4b** (ngữ cảnh 32K token, chạy qua Ollama, không cần `transformers/torch` như MedCPT) sau một phép so sánh nhỏ: $n=1273$ chưa chạy, nhưng trên $n=40$ câu test (qwen2.5:7b), qwen3-embedding:4b đạt **0.600** độ chính xác - bằng đúng V0 (0.600, tức truy hồi không kéo tụt) và cao hơn embedder ban đầu (0.550) 5 điểm, đồng thời nhanh hơn $\sim 20\%$ (7.80s so với 9.68s/câu). Lợi thế đó sau đó được áp dụng cho toàn bộ V2-V5. Ngữ liệu nhúng lại là 30.000 câu MedMCQA đầu tiên (một tập con của 182.822 câu gốc) vào một collection riêng (`knowledge_medmcqa_qwen3`); phần còn lại đang được nhúng nốt để khớp quy mô ngữ liệu gốc. **[CẦN LÀM: Đo lại V1-V5 (qwen3-embedding:4b) ở $n=1273$ khi ngữ liệu nhúng lại xong toàn bộ 182.822 câu.]**



Hình 8: Hạ tầng truy hồi: từ ngữ liệu MedMCQA, qua embedding và ChromaDB, đến công cụ `search_medmcqa` và bằng chứng thô.



Hình 9: Từ bằng chứng thô (Hình 8), qua bước digest + tự chấm độ tin cậy, đến luật tin cậy bất đối xứng - dùng chung cho bước tự quyết định của V1 và Node 4 (decider) của V2-V4.

V2 tách bốn việc mà tác tử đơn của V1 làm nối tiếp (hiểu ca bệnh, tìm kiếm, lập luận, quyết định) thành bốn nút, hai trong số đó (`search`, `reasoning`) chạy đồng thời. `search` và `reasoning` được nối trực tiếp từ `understand` (không nối tiếp nhau), nên LangGraph chạy chúng trong cùng một superstep; `answer` chờ CẢ HAI xong mới chạy (nối bằng `add_edge([search, reasoning], answer)`), dạng danh sách, để `answer` chỉ chạy đúng một lần). **4 tác tử, 4 lượt gọi LLM/câu** (`understand`, `evidence-digest`, `clinical-reasoner`, `decider`) - không còn hội đồng nhiều chuyên gia tranh luận theo góc nhìn (thiết kế trước, kiểu MedAgents [9]); thay vào đó hai nhánh độc lập KHÔNG thấy đầu ra của nhau, và `decider` dùng một luật tường minh (chứ không phải một bước tổng hợp tự do) để quyết nhánh nào được tin. **[CẦN LÀM: Đo V2 mới ở quy mô lớn; số cũ ở §5 đo trên thiết kế hội đồng trước đó, không áp dụng được cho thiết kế này.]**

Lựa chọn ngữ liệu. MedMCQA (Pal et al. [7], 182.822 câu thi đã giải kèm giải thích) là ngữ liệu truy hồi *duy nhất* trong hệ thống hiện tại, dùng cho toàn bộ V1-V5 - tức truy hồi các ví dụ mẫu đã giải (tương tự Medprompt [6]), không phải văn bản tham khảo. Bộ kiểm chứng của V5 (§3.4) cũng không có công cụ tra cứu riêng ngoài kho tiến hoá của chính nó; chỉ bộ kiểm chứng của V4 mới có một nguồn độc lập với MedMCQA - Wikipedia trực tuyến (§3.4).

3.3 Thiết kế bộ nhớ

Đề bài liệt kê bộ nhớ ngắn hạn và dài hạn là các thành phần bắt buộc. **[CẦN LÀM: Đổi định nghĩa so với bản báo cáo trước: bộ nhớ ngắn hạn nay thuộc về V4/V5 (bộ kiểm chứng), không còn ở V3; bộ nhớ dài hạn nay ĐÃ XÂY (trước đây báo là chưa) - hai kho riêng biệt, xem dưới. Chưa đo ở quy mô lớn.]**

Bộ nhớ ngắn hạn (trong một câu hỏi). Xuất hiện ở V4/V5 (Hình 6, §3.1) - V3 không có bộ kiểm chứng nên không cần đọc lại gì. **Trạng thái: đã xây, ở dạng hẹp.** Thay vì một tác tử/kho bộ nhớ riêng, bộ nhớ ngắn hạn tận dụng `QASState` - trạng thái dùng chung mà LangGraph truyền qua các nút: `case_understanding` (bản tóm tắt ca bệnh của Node 1), `evidence`, và `clinical_report` (báo cáo của Node 3) đều đã nằm sẵn trong đó. Khi `memory=True` (V4/V5), bộ kiểm chứng (Node 5) đọc thêm `case_understanding` khi audit đáp án, thay vì chỉ đọc báo cáo lập luận; không nút nào khác cần bật cờ này, vì các nút khác vốn đã đọc thẳng những gì nút trước ghi ra - `memory` chỉ mở rộng đúng MỘT điểm đọc thêm cho bộ kiểm chứng, không thêm tác tử hay lượt gọi LLM nào.

Bộ nhớ dài hạn (episodic, xuyên câu hỏi): ĐÃ XÂY, hai kho tách biệt. Cả hai dùng chung một `SqliteStore` của LangGraph (`data/longterm/lessons.sqlite`, sống sót qua `process`, `graph/longterm.py`), phân biệt bằng namespace gốc, không phải bằng tệp riêng:

- **Kho chung** (`medqa-lessons`, V3-V5): `decider` (Node 4) `recall` bài học từ những câu TUỒNG TỰ đã gặp trước - xếp hạng bằng số từ chủ đề trùng nhau (không phải embedding, để giữ chi phí thấp và không cần một collection thứ hai) - rồi `remember` một bài học sau khi chốt, cho MỌI câu, đúng lẫn sai.
- **Kho tiến hoá** (`medqa-mistakes`, V5): chỉ chứa bài học từ những câu hệ thống từng trả lời SAI. Bộ kiểm chứng (Node 5) chỉ `recall` từ kho này, không bao giờ ghi và không bao giờ thấy đáp án đúng; Node 6 (`distill_mistake`), chạy SAU Node 5, là nút DUY NHẤT đọc đáp án đúng, và chỉ ghi một bài học khi đáp án cuối cùng sai (§3.1).

Chống rò rỉ dữ liệu (contamination): namespace mỗi kho gồm cả `split` (train/test), và mặc định (`long_term_read_only=True`) là CHỈ ĐỌC - ghi vào kho là một hành động tường minh

Bộ nhớ ngắn hạn — mọi nút đọc/ghi qua một QASpace dùng chung

Không tác tử/kho bộ nhớ riêng — `memory=True` chỉ mở thêm một quyền đọc cho Node 5

Trường trong QASpace	Ghi bởi	Đọc bởi
item	(đầu vào)	mọi nút
query	Node 1 (understand)	Node 2 (search)
evidence	Node 2 (search)	Node 4 (decider)
clinical_report	Node 3 (reasoning)	Node 4 (decider), Node 5 (verifier)
case_understanding	Node 1 (understand)	Node 3 (reasoning); Node 5 (verifier) CHỈ khi <code>memory=True</code> (V3)
answer, rationale	Node 4 (decider)	ghi đè bởi Node 5, nếu có (V3)

`memory=True` mở đúng MỘT ô được tô (Node 5 đọc `case_understanding`) — không thêm tác tử, không thêm lượt gọi LLM.

Không có bộ nhớ dài hạn (episodic) xuyên câu hỏi trong bảng này — mỗi hàng chỉ tồn tại trong một lượt (một câu hỏi).

Hình 10: Bộ nhớ ngắn hạn là QASpace dùng chung: mỗi trường được một nút ghi và một hoặc nhiều nút khác đọc lại; hàng tô vàng là ô duy nhất `memory=True` mở thêm (sơ đồ minh họa ở dạng cũ - **[CẦN LÀM: cập nhật cho khớp bộ kiểm chứng V4/V5]**).

(`--remember`), thực hiện trên tập train, tách khỏi lượt chạy tính điểm trên test. Nếu không, bài học của câu thứ N đang được chấm có thể rò sang câu thứ $N+k$ trong CÙNG lượt chấm đó, làm tăng ảo độ chính xác vì lý do không liên quan tới chất lượng lập luận.

3.4 Thiết kế bộ kiểm chứng

[CẦN LÀM: Đổi định nghĩa so với bản báo cáo trước: bộ kiểm chứng nay chỉ xuất hiện ở V4/V5 (không còn ở V3); có HAI thiết kế bộ kiểm chứng khác nhau, không còn một thiết kế duy nhất.] V4 và V5 dùng CHUNG Node 5 (`report-verifier`): đọc thẳng báo cáo lập luận của Node 3 (`state["clinical_report"]`), đáp án và giải thích ngắn mà Node 4 vừa chốt, và (`memory=True`) cả bản tóm tắt ca bệnh của Node 1 - không cần một tác tử/lượt gọi LLM riêng để “nhớ lại” những thứ đó, vì chúng đã nằm sẵn trong trạng thái đồ thị dùng chung. Nó chỉ kiểm nhất quán: nếu đáp án được chọn bị chính báo cáo lập luận đánh giá RULED OUT (hoặc một phương án khác được đánh giá rõ SUPPORTED), đó là dấu hiệu đáng ngờ và nó chỉnh lại đáp án cho khớp với những gì báo cáo đã kết luận. Để giữ luồng bền vững, một lỗi trích xuất ở Node 5 sẽ lùi về đáp án của Node 4 thay vì phát ra `None` không hợp lệ.

Điểm khác nhau DUY NHẤT giữa V4 và V5 là nguồn bằng chứng bổ sung khi báo cáo của Node 3 không đủ để quyết:

- **V4 - Wikipedia trực tuyến.** Gọi `search_wikipedia` (MediaWiki API, một lệnh gọi mạng, có timeout, trả về "" khi lỗi mạng - suy biến về đúng hành vi V3, không làm hỏng lượt chạy). Đây là nguồn bằng chứng ĐỌC LẬP đầu tiên trong toàn hệ thống không bắt nguồn từ MedMCQA.
- **V5 - kho tiến hoá.** `recall_mistakes` từ kho `medqa-mistakes` (§3.3) - bài học từ những câu TƯƠNG TỰ hệ thống từng trả lời sai trước đó. Bộ kiểm chứng không bao giờ gọi công cụ mạng và không bao giờ thấy đáp án đúng của câu đang xử lý.

V5 còn có thêm **Node 6 - distill_mistake**, chạy SAU Node 5: nút DUY NHẤT trong đồ thị

đọc `item.answer_idx` (đáp án đúng). Nếu đáp án cuối cùng ĐÚNG, nó không làm gì (không tốn lượt gọi LLM, không dùng kho). Nếu SAI, nó gọi một tác tử `mistake-analyst` - được cho biết cả đáp án sai lẫn đáp án đúng, việc mà Node 5 không bao giờ được biết - để đúc kết một bài học sửa sai, rồi ghi vào kho tiến hoá cho các câu tương lai. Nút này KHÔNG BAO GIỜ được phép sửa `state["answer"]` / `["rationale"]`: việc của nó là ghi cho các câu hỏi TƯƠNG LAI, không phải ảnh hưởng câu hỏi HIỆN TẠI đang được chấm.

Phần đóng góp riêng của mỗi thiết kế bộ kiểm chứng được đo trực tiếp bằng tương phản $V4 - V3$ (bộ kiểm chứng Wikipedia) và $V5 - V3$ (bộ kiểm chứng kho tiến hoá): vì cả ba biến thể dùng chung đúng một đồ thị nền ($V3$, §3.1), mỗi tương phản chỉ khác đúng Node 5 (và với $V5$, cả Node 6) - không lẫn với ngữ liệu, embedder, hay số tác tử. **[CẦN LÀM: Đo $V4 - V3$ và $V5 - V3$ khi cả ba được đo ở quy mô lớn; hiện chỉ có số liệu $n=300$ cho định nghĩa CŨ của $V3$ (§5), chưa áp dụng được cho định nghĩa mới.]**

3.5 Tối ưu hoá tiến hoá

Không dùng trong hệ giữa kỳ này. Đề bài liệt kê nó là tùy chọn; Nhóm chủ động bỏ qua để giữ khảo sát ablation dễ diễn giải và dời việc tìm kiếm prompt/pipeline sang phần sau. *Lưu ý thuật ngữ*: dùng nhằm mục này với “kho tiến hoá” (§3.3, $V5$) - kho tiến hoá chỉ là một kho bài học tích lũy dần qua các câu sai, không phải một thuật toán tối ưu hoá kiểu di truyền/tiến hoá cho prompt hay pipeline.

Đáp án + giải thích. Đề bài yêu cầu một lựa chọn *cộng với* một giải thích ngắn. Mọi biến thể đều trả về cả hai qua một giao diện duy nhất `answer(item) → AnswerResult(answer, rationale)`, và bản ghi từng câu mang theo phần giải thích để tệp dự đoán có nó mà không cần chạy lại. Chỉ nút *cuối cùng* ghi đáp án hướng tới người dùng mới bị giới hạn 30 từ (Node 4 và Node 5, các biến thể có bộ kiểm chứng): Node 1 (tóm tắt ca bệnh), Node 2 (tóm bằng chứng) và Node 3 (báo cáo lập luận) chủ ý *không* bị giới hạn độ dài, vì đó là đầu vào nội bộ cho Node 4/5 đọc, không phải đầu ra hướng tới người dùng - giới hạn từ ở đó sẽ chỉ làm cùn lập luận mà các nút này tồn tại để tạo ra. Node 6 (`distill_mistake`, $V5$) không thuộc nhóm nào ở trên: nó không ghi đáp án hướng tới người dùng và cũng không phải đầu vào cho một nút khác trong CÙNG câu hỏi - đầu ra của nó (một dòng `Lesson:`) chỉ đi vào kho tiến hoá, để CÂU HỎI KHÁC trong tương lai đọc lại.

4 Thiết lập thực nghiệm

4.1 Mô hình và prompt

Mô hình trả lời mặc định là **Qwen2.5** [8] (Ollama, có gọi công cụ): `qwen2.5:7b` cho mọi biến thể và mọi con số được báo cáo. Mọi vai chạy ở **hiệu suất 0**: ở nhiệt độ mặc định, chênh lệch $V1 - V0$ dao động giữa các lần chạy lớn hơn cả bản thân hiệu ứng, nên tính tất định là bắt buộc để McNemar ghép cặp có ý nghĩa. Các nhà cung cấp đám mây (`anthropic:`, `openai:`, `google:`, `openrouter:`) được hỗ trợ qua cùng một đặc tả `provider:model`.

System prompt của baseline ($V0$) và lượt người dùng cho câu trắc nghiệm, nguyên văn (giữ tiếng Anh vì đây là prompt vận hành thực tế):

```
System (roles.py:BASELINE):
You are an expert physician answering a medical board (USMLE)
multiple-choice question. Choose the single best answer, with a
brief justification.
```

```
User turn (qa/mcq.py:format_mcq):  
{question}
```

```
A. {option 0}  
B. {option 1}  
C. {option 2}  
D. {option 3}
```

```
Reason briefly, then on the LAST line write 'Answer: X' where X is  
A, B, C, or D.
```

`parse_choice` trích đáp án từ **Answer: X** hoặc **answer is X** (không phân biệt hoa thường), dự phòng bằng chữ cái A|B|C|D đọc lập đầu tiên; phản hồi không trích được trả về **None** và bị tính là *không hợp lệ*, không bao giờ bị tính nhầm thành câu trả lời sai. Tỷ lệ không hợp lệ đo được là **0.0%** xuyên suốt.

4.2 Cấu hình truy hồi

Bảng 2: Thiết lập truy hồi. Cùng ngữ liệu, embedder và collection cho toàn bộ V1-V5 kể từ đợt A/B ở §3.2.

Thiết lập	V1-V5
Ngữ liệu	MedMCQA: 182,822 câu thi đã giải
Collection	<code>knowledge_medmcqa_qwen3</code>
Kho vector	Chroma, cosine / HNSW, lưu trữ bền
Embedder	<code>qwen3-embedding:4b</code> (32K token)
Cái được nhúng	chỉ phần câu hỏi (đáp án/giải thích/chủ đề là metadata)
Top- <i>k</i>	5, không cố (threshold=0.0)
Cổng độ tương đồng	không dùng ở bất kỳ đâu trong hệ thống hiện tại

Embedder được chọn vì có ngữ cảnh dài: Query encoder của các embedder bất đối xứng thường gặp (vd. MedCPT) cắt truy vấn ở khoảng **64 token**, quá ngắn cho một tình huống USMLE mà câu hỏi thực nằm ở cuối, trong khi `qwen3-embedding` (32K token) nhúng trọn cả tình huống. Thiết kế trước dùng một cổng độ tương đồng bằng số, hiệu chỉnh riêng theo embedder; cổng số đó nay không còn ở bất kỳ đường truy hồi nào trong hệ thống: việc quyết định có tin bằng chứng hay không chuyển hẳn sang bước chốt đáp án, dựa trên nhãn tin cậy HIGH/MEDIUM/LOW mà chính mô hình tự chấm (§3.2) - một cổng số học phía trên nhãn tự chấm đó là dư thừa. Bộ kiểm chứng của V3 không có công cụ tra cứu riêng (§3.4), nên bảng trên là toàn bộ cấu hình truy hồi trong hệ thống.

4.3 Phần cứng / thiết lập API

- **Phần cứng:** máy Mac Apple Silicon, 48 GB bộ nhớ hợp nhất, chạy Ollama cục bộ.
- **Seed ngẫu nhiên:** `bootstrap_ci(seed=0)`; sinh văn bản là greedy ở nhiệt độ 0 nên không áp dụng seed lấy mẫu.
- **Lưu ý khi đo độ trễ:** một câu khởi động tổng hợp được chạy trước (khởi động mô hình người sẽ tính cho câu chấm điểm đầu tiên ~1.9s so với ~0.3s ở trạng thái ổn định); các mô hình không phải mục tiêu bị dừng trước một lần chạy đo giờ để tránh tranh chấp GPU.

- **Dữ liệu:** openlifescienceai/medqa (train 10,178 / validation 1,272 / **test 1,273**), lưu cache cục bộ và nạp không cần mạng.

Tái lập một lần chạy.

```
HF_HUB_OFFLINE=1 PYTHONPATH=src .venv/bin/python -m agent_hospital -v V0 -n 150
```

5 Kết quả đánh giá

Chỉ số chính là độ chính xác, $\text{Độ chính xác} = (\text{số dự đoán đúng})/(\text{số câu hỏi})$. Nhóm báo cáo thêm tỷ lệ phản hồi không hợp lệ, mức tăng độ chính xác so với V0, Thắng/Thua/Hòa so với baseline, khoảng tin cậy bootstrap 95%, kiểm định ghép cặp McNemar, độ trễ và số token.

Trạng thái đo lường. Mục này báo cáo một lần chạy trên **300 câu đầu của tập test** ($n=300/1273$, qwen2.5:7b, nhiệt độ 0) cho cả bốn biến thể V0-V3, trên kiến trúc *hiện tại* (thiết kế bốn nút + luật tin cậy bất đối xứng, §3.1) và ngữ liệu truy hồi hiện tại (qwen3-embedding:4b cho V1-V3, §3.2) - không phải thiết kế hội đồng cũ mà các số liệu trước đây trong báo cáo mô tả. V4 chưa được đo. **[CẦN LÀM: Chạy trên toàn bộ $n=1273$ để có số liệu chính thức; $n=300$ ở đây đã đủ lớn để McNemar có ý nghĩa cho V2 và V2 vs V1, nhưng V3 vs V0 và V3 vs V2 vẫn chưa đạt ngưỡng có ý nghĩa thống kê (xem dưới).]**

5.1 Bảng chính (main board): 300 câu đầu của tập test, qwen2.5:7b

Bảng 3: Kết quả trên 300 câu đầu của tập test, qwen2.5:7b, nhiệt độ 0.

Variant	Accuracy	95% Bootstrap CI	Invalid	Latency/query	Tokens/query
V0: LLM trực tiếp	0.613 (184/300)	[0.557, 0.670]	0.0%	1.71 s	489
V1: RAG-only (agen-tic)	0.597 (179/300)	[0.540, 0.650]	0.0%	4.64 s	2880
V2: Đa tác tử (4 nút)	0.667 (200/300)	[0.613, 0.717]	0.0%	19.13 s	4371
V3: V2 + bộ kiểm chứng	0.650 (195/300)	[0.597, 0.703]	0.0%	22.31 s	5905

Tỷ lệ không hợp lệ là **0.0% ở cả bốn biến thể** - khác hẳn thiết kế trước, nơi V1 (agentic, thiết kế cũ) có 3.85% không hợp lệ vì vòng lặp gọi công cụ đôi khi không hội tụ về một chữ cái; luật quyết định hiện tại (tin cậy bất đối xứng, không bắt buộc chỉ trả lời một chữ cái) giải quyết hẳn vấn đề đó. V1 **thấp hơn V0** (-1.7 điểm, khoảng tin cậy chồng lấn gần như toàn bộ - xem McNemar dưới), nhất quán với mọi lần đo retrieval-only trong dự án. V2 **cao nhất** (+5.3 điểm so với V0), và V3 (+3.7 điểm) đứng giữa V0 và V2 - thấp hơn V2 một chút chứ không cao hơn.

5.2 So sánh với baseline (so sánh ghép cặp)

5.3 Phân tích Thắng/Thua/Hòa

Cột T/T/H đếm theo từng câu: biến thể sửa được lỗi của baseline (thắng), làm hỏng một câu baseline vốn đúng (thua), hay trùng khớp (hòa). **V2 vs V0:** 36 thắng so với 20 thua - V2 sửa được gần gấp đôi số câu nó làm hỏng, và với 56 câu bất đồng (discordant), McNemar cho $p=0.044$: đây là cải thiện có ý nghĩa thống kê đầu tiên trong toàn bộ khảo sát. **V1 vs V0:** 29/34, gần như

Bảng 4: So sánh ghép cặp trên 300 câu đầu của tập test, qwen2.5:7b. Tính trực tiếp từ bản ghi từng câu.

Comparison	Delta	W/L/T	McNemar p
V1 vs V0	-1.7 điểm	29/34/237	0.615 (không ý nghĩa)
V2 vs V0	+5.3 điểm	36/20/244	0.044 (có ý nghĩa)
V3 vs V0	+3.7 điểm	34/23/243	0.185 (không ý nghĩa)
V3 vs V2 (hiệu ứng kiểm chứng)	-1.7 điểm	3/8/289	0.227 (không ý nghĩa)
V2 vs V1 (đa tác tử so với truy hồi)	+7.0 điểm	48/27/225	0.020 (có ý nghĩa)

cân bằng - truy hồi sửa một số câu nhưng làm hỏng nhiều hơn, ròng lại là một khoản lỗ nhỏ không đáng kể. **V3 vs V2**: chỉ 11 câu bất đồng trên 300 (3 thắng, 8 thua cho V3) - bộ kiểm chứng hiếm khi đổi đáp án của Node 4, và khi đổi thì đổi sai nhiều hơn đổi đúng, dù mẫu quá nhỏ để kết luận chắc chắn ($p=0.227$). **Chưa có bằng chứng ở quy mô này rằng bộ kiểm chứng cải thiện độ chính xác** - đây là phát hiện ngược với kỳ vọng ban đầu của Nhóm, và được báo cáo trung thực thay vì diễn giải có lợi cho thiết kế.

5.4 Chi phí và độ trễ

Chi phí đo bằng *độ trễ* và *số token* (đầu vào + đầu ra) cho mô hình cục bộ (không có phí API). V0: **1.71s/câu, 489 token/câu**. V1 (agentic, qwen3-embedding:4b): **4.64s/câu** ($\sim 3 \times V0$), **2880 token/câu** ($\sim 6 \times V0$) - vòng lặp gọi công cụ (tìm rồi trả lời) sinh nhiều token hơn một lượt trả lời trực tiếp, nhưng nhanh hơn hẳn thiết kế cũ (trước đây 3.98s ở phiên bản panel, với embedder ban đầu). V2 (4 nút: hiểu ca bệnh + tìm kiếm + lập luận + quyết định): **19.13s/câu** ($\sim 11 \times V0$), **4371 token/câu** ($\sim 9 \times V0$) - bốn tác tử tuần tự (Node 2 và Node 3 chạy đồng thời nhưng Node 1 và Node 4 vẫn tuần tự), nên chi phí cộng dồn theo số lượt gọi LLM. V3 (V2 + bộ kiểm chứng): **22.31s/câu** ($\sim 3.2s$ hơn V2, đúng bằng chi phí một lượt gọi LLM thêm của Node 5), **5905 token/câu**. Vì bộ kiểm chứng không cho thấy cải thiện độ chính xác đáng kể ở quy mô này (§5.2), chi phí thêm $\sim 17\%$ độ trễ và $\sim 35\%$ token của V3 so với V2 hiện chưa được biện minh bằng bằng chứng - một điểm cần theo dõi khi đo ở quy mô lớn hơn. **[CẦN LÀM: Chạy V4 (V2 không có Node 5, cùng cấu hình) để đối chiếu trực tiếp với V3; hiện V4 chưa được đo. Chạy lại toàn bộ ở $n=1273$ khi có thời gian, đặc biệt để xác nhận hoặc bác bỏ ý nghĩa thống kê của V3 vs V0 và V3 vs V2, cả hai hiện đang ở ranh giới ($p=0.185$ và $p=0.227$).]**

6 Phân tích lỗi

Mục này dùng tệp dự đoán từng câu của V0-V3 (data/eval_runs/*.jsonl, $n=300$, xem §5) - không còn chỉ là mô tả định tính.

Tỷ lệ không hợp lệ là 0.0% trên cả bốn biến thể, nên mọi lỗi đều là chọn sai lựa chọn thực sự, không phải lỗi trích xuất. Hai mẫu hình hiện rõ trong dữ liệu ghép cặp:

- **Câu trung tính với truy hồi.** V1 vs V0: 29 thắng, 34 thua, 237 hòa (§5) - trên đa số câu (79%), truy hồi không đổi đáp án, nhất quán với việc MedQA chủ yếu đòi hỏi lập luận chứ không phải tra cứu dữ kiện.
- **Câu nhạy với cấu trúc lập luận.** V2 vs V1: 48 thắng, 27 thua (§5) - tách hiểu ca bệnh / tìm kiếm / lập luận / quyết định thành bốn nút làm đổi quyết định trên nhiều câu hơn hẳn

so với chênh lệch do riêng truy hồi gây ra, khu trú phần lớn mức tăng vào khâu cấu trúc lập luận chứ không phải bằng chứng truy hồi (vẫn cùng một nguồn MedMCQA).

Bảng 5: Phân tích lỗi với ví dụ thật từ tập dự đoán $n=300$ (V0 vs V2).

Loại lỗi	Mã câu	Ghi chú
V2 lập luận chi tiết nhưng chốt sai, trong khi V0 (một lượt) tình cờ đúng	test-00012	V0 chọn đúng (B); Node 3 của V2 xây một chuỗi lập luận cụ thể dẫn tới u xơ thần kinh loại 2, decider theo lập luận đó và chọn D (sai).
Bị phương án nghe hợp lý nhưng sai đánh lừa	test-00013	V0 chọn đúng (D, sa van hai lá); V2 lập luận nghiêng về bệnh cơ tim phì đại tắc nghẽn (HOCM) và chọn A (sai) - một chẩn đoán phân biệt hợp lý nhưng không khớp đặc điểm tiếng thổi trong đề bài.
V2 sửa đúng nhờ tri thức chuyên biệt hơn	test-00003	V0 chọn sai (B, nhiễm khuẩn vùng chậu); Node 3 của V2 xác định đúng đặc điểm vi khuẩn học (khuẩn lạc hồng trên môi trường MacConkey) và chọn D (đúng).
V2 sửa đúng nhờ bám sát đúng câu hỏi được hỏi	test-00024	V0 chọn sai (B, tràn khí màng phổi áp lực); V2 nhận ra câu hỏi thực sự là bước xử trí tiếp theo chứ không phải chẩn đoán, chọn A (đặt nội khí quản, đúng).

[CẦN LÀM: Bảng trên mới liệt kê 4 ví dụ minh họa, chưa phải phân loại lỗi có hệ thống trên toàn bộ các câu bất đồng; có thể mở rộng bằng cách phân loại tự động (vd. LLM-judge) trên tập dự đoán khi có thời gian.]

7 Thảo luận

Bài học nổi bật là: **trên MedQA, đòn bẩy là cấu trúc lập luận, không phải truy hồi**. Đây không phải lỗi tinh chỉnh: Nhóm đã đổi sang embedder ngữ cảnh dài hơn (qwen3-embedding:4b), bỏ cổng độ tương đồng để thay bằng nhãn tin cậy tự chấm, và cho phép tác tử tự quyết định khi nào/tìm gì thay vì một pipeline cố định - mà truy hồi vẫn không làm thay đổi độ chính xác một cách có ý nghĩa (-1.7 điểm so với V0, $p=0.615$). Y văn cũng ghi nhận điều tương tự: mức tăng của RAG không đồng đều giữa các bộ dữ liệu: mạnh trên các bộ nặng về tra cứu (PubMedQA, BioASQ) và yếu trên các bộ nặng về lập luận/trí nhớ như MedQA, vì một mô hình đủ mạnh vốn đã nắm sẵn tri thức và phần việc còn lại là *tích hợp*. Kết quả V2 của Nhóm là mặt tích cực của câu chuyện này: tách hiểu ca bệnh / tìm kiếm / lập luận / quyết định thành bốn nút, trên *cùng* bằng chứng truy hồi mà V1 đã có, tạo ra mức cải thiện có ý nghĩa thống kê đầu tiên trong toàn bộ khảo sát ($+5.3$ điểm so với V0, $p=0.044$; $+7.0$ điểm so với V1, $p=0.020$). Điều bất ngờ là bước tiếp theo - thêm bộ kiểm chứng (V3) - *chưa* cho thấy cải thiện thêm ở quy mô này: V3 thấp hơn V2 một chút (-1.7 điểm, không ý nghĩa), dù tốn thêm đáng kể độ trễ và token. Có thể đọc theo hai hướng: (i) lợi ích của kiểm chứng thật sự nhỏ hoặc bằng không trên MedQA khi lập luận nền đã đủ tốt (Node 3 của V2 đã tự chấm độ tin cậy, nên phần việc “kiểm tra lại” phần lớn đã làm rồi), hoặc (ii) $n=300$ chưa đủ lớn để phân biệt một hiệu ứng nhỏ với nhiễu ngẫu nhiên - hai giả thuyết này cần đo ở $n=1273$ để phân định.

8 Hạn chế

- **Quy mô.** V0-V3 mới đo ở $n=300/1273$ câu test; V4 chưa được đo. Ở quy mô này, V3 vs V0 ($p=0.185$) và V3 vs V2 ($p=0.227$) chưa đạt ngưỡng có ý nghĩa thống kê - có thể đổi chiều

khi đo trên toàn bộ 1273 câu.

- **Thiếu thành phần.** Bộ nhớ dài hạn (§4.5) chưa được xây; bộ nhớ ngắn hạn (§4.4) chỉ mới ở dạng hẹp (đọc trạng thái đồ thị dùng chung, chỉ phục vụ bộ kiểm chứng), nên hệ thống hiện tại vẫn chưa phải “hệ đầy đủ” theo đề bài.
- **Chưa có chi phí tiền.** Đã có độ trễ và số token (§5); chưa quy đổi ra chi phí tiền (không áp dụng cho mô hình cục bộ, nhưng cần nếu so sánh với API đám mây).
- **Bộ kiểm chứng chưa chứng minh được lợi ích.** V3 (V2 + bộ kiểm chứng) thấp hơn V2 một chút (-1.7 điểm, không ý nghĩa) trong khi tốn thêm $\sim 17\%$ độ trễ và $\sim 35\%$ token - ngược với kỳ vọng thiết kế ban đầu của Nhóm; cần đo ở quy mô lớn hơn trước khi kết luận.
- **Cổng do mô hình tự chấm chưa được kiểm định.** Cổng độ tương đồng bằng số đã được thay bằng nhãn tin cậy tự chấm của chính mô hình (§3.2); độ tin cậy của chính nhãn tự chấm đó (so với một ngưỡng số khách quan) chưa được đo.
- **Tính tắt định trên đám mây.** Nhiệt độ 0 được đặt, nhưng các API lưu trữ không bảo đảm tính tái lập như giải mã greedy cục bộ; cổng golden được ghim vào một mô hình cục bộ.

9 Kết luận

Nhóm đã xây một hệ đa tác tử y khoa điều khiển bằng cấu hình dưới dạng một thang biến thể nằm bậc (ablation ladder) trên MedQA-USMLE, trong đó mỗi bậc khác nhau đúng một thành phần và được chấm bởi một bộ đo ghép cặp, có nhận thức về ý nghĩa thống kê. Bằng chứng đo được ở $n=300$ rõ ràng ở điểm cốt lõi: riêng truy hồi không cải thiện độ chính xác MedQA (V1 vs V0: -1.7 điểm, không ý nghĩa), trong khi tách cấu trúc lập luận thành bốn nút thì có (V2 vs V0: $+5.3$ điểm, $p=0.044$) - cải thiện có ý nghĩa thống kê đầu tiên trong toàn bộ khảo sát. Thêm bộ kiểm chứng (V3) chưa cho thấy lợi ích thêm ở quy mô này (V3 vs V2: -1.7 điểm, không ý nghĩa), một kết quả bất ngờ cần đo ở quy mô lớn hơn để phân định. Các bước tiếp theo trước mắt là (i) chạy toàn bộ tập test chính thức ($n=1273$) để xác nhận hoặc bác bỏ ý nghĩa thống kê của V3 vs V0 và V3 vs V2, (ii) đo V4 (hiện chưa đo) để tách riêng hiệu ứng bộ kiểm chứng (V3 – V4) một cách trực tiếp, và (iii) thêm bộ nhớ dài hạn và theo dõi chi phí tiền để khép các khoảng trống còn lại so với đề bài. Hệ thống được thiết kế sao cho mỗi việc trên là một thay đổi cấu hình hoặc bộ đo chứ không phải viết lại.

A Danh mục kiểm tra tái lập

Mục	Giá trị
Mô hình	<code>qwen2.5:7b</code>
Nhiệt độ	0 (mọi vai)
Prompt	nguyên văn trong <code>roles.py</code> ; baseline xem §4
Truy hồi	embedder <code>qwen3-embedding:4b</code> (V1-V5), Chroma cosine/HNSW, $\text{top-}k=5$, không cổng
Seed ngẫu nhiên	<code>bootstrap_ci(seed=0)</code> ; còn lại là giải mã greedy
Phần cứng	máy Mac Apple Silicon, 48 GB, Ollama cục bộ
Dữ liệu	<code>openlifescienceai/medqa</code> (test $n=1273$; báo cáo này dùng 300 câu đầu), cache ngoại tuyến
Điểm vào	<code>build_variant(id, model=, temperature=0, **overrides)</code>

Tài liệu tham khảo

- [1] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. volume 11, page 6421, 2021.
- [2] Qiao Jin, Won Kim, Qingyu Chen, Donald C Comeau, Lana Yeganova, W John Wilbur, and Zhiyong Lu. Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval. *Bioinformatics*, 39(11), 2023.
- [3] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeonhoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. Mdagents: An adaptive collaboration of llms for medical decision-making. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.15155.
- [4] LangChain. Langgraph: Building stateful, multi-actor applications with llms. <https://langchain-ai.github.io/langgraph/>, 2024.
- [5] Junkai Li, Yunghwei Lai, Weitao Li, Jingyi Ren, Meng Zhang, Xinhui Kang, Siyu Wang, Peng Li, Ya-Qin Zhang, Weizhi Ma, and Yang Liu. Agent hospital: A simulacrum of hospital with evolvable medical agents. *arXiv preprint arXiv:2405.02957*, 2024.
- [6] Harsha Nori, Yin Tat Lee, Sheng Zhang, Dean Carignan, Richard Edgar, et al. Can generalist foundation models outcompete special-purpose tuning? case study in medicine. *arXiv preprint arXiv:2311.16452*, 2023.
- [7] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on Health, Inference, and Learning (CHIL)*, 2022.
- [8] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- [9] Xiangru Tang, Anni Zou, Zhuosheng Zhang, Ziming Li, Yilun Zhao, Xingyao Zhang, Arman Cohan, and Mark Gerstein. Medagents: Large language models as collaborators for zero-shot medical reasoning. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2024. arXiv:2311.10537.
- [10] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang. Benchmarking retrieval-augmented generation for medicine. *Findings of the Association for Computational Linguistics (ACL)*, 2024.